

Lecture Notes on Topos-Theoretic Frameworks for Neural Networks and LLMs

June 26, 2026

Abstract

These notes give a textbook-style introduction to the mathematical ideas behind recent claims that certain neural-network or large-language-model structures can be placed inside categorical, sheaf-theoretic, or topos-theoretic frameworks. The goal is not to present these claims as established machine-learning theorems. The goal is to explain, carefully and in a unified notation, what the categorical words mean, how they are used, and what has to be checked before a statement such as “a class of neural networks forms a topos” becomes a mathematical theorem. We begin with categories, functors, natural transformations, limits, adjunctions, presheaves, Yoneda, sheaves, sites, Grothendieck topoi, internal logic, and stacks. We then read the existing topos-AI proposals as case studies in mathematical modelling.

1 The Central Question

A neural network is often written as a parametrized map

$$N_\theta : X \longrightarrow Y$$

or as a composite of layers

$$N_\theta = f_{L,\theta_L} \circ f_{L-1,\theta_{L-1}} \circ \cdots \circ f_{1,\theta_1}.$$

This notation is correct: after the parameters θ are fixed, the network really does define a function from an input space to an output space. The point is that this notation mainly records the global input-output behaviour. If one looks only at the resulting function, much of the implementation-level structure has been compressed away.

For example, two networks may define the same function $X \rightarrow Y$ while using different intermediate representations. Conversely, two intermediate representations may fail to be literally equal while differing only by an invertible change of coordinates. If at layer l one changes feature coordinates by

$$h_l : V_l \longrightarrow V_l$$

and adjusts the neighbouring layers by the corresponding conjugations, the global composite N_θ may remain unchanged. Thus the single notation

$$N_\theta : X \rightarrow Y$$

does not say which representations should count as different coordinate descriptions of the same structure.

Similarly, the layer-composite formula displays an order of computation, but it does not by itself define what “local” means. Locality may refer to image patches, token subwindows, layers, attention heads, task contexts, or semantic fragments. To ask whether local information glues to global information, one must first specify a category of local pieces and a notion of cover. The ordinary function notation does not contain these choices.

Thus the point is not that the usual notation is wrong. The point is that it is external: it treats the network as one global function, without explicitly recording local data, restriction maps, coordinate equivalences, context-dependence, or local-to-global gluing rules. Sheaf and topos language is an attempt to make these extra structures part of the mathematical object.

Topos-theoretic approaches start from the opposite direction. They ask for a category of contexts $\mathcal{C}$, a notion of covering J , and a category of sheaves

$$\mathbf{Sh}(\mathcal{C}, J).$$

The object $\mathbf{Sh}(\mathcal{C}, J)$ is then interpreted as a universe of context-dependent objects satisfying a local-to-global principle.

Principle 1.1 (Responsible reading). *When a paper says that neural networks, transformers, or LLMs have a topos-theoretic structure, one should ask four questions.*

1. *What is the category $\mathcal{C}$?*
2. *What are its morphisms?*
3. *What is the topology J , i.e. what counts as a cover?*
4. *What exactly is the sheaf, topos, stack, or universal construction being used?*

Without answers to these questions, the word “topos” is a metaphor rather than a theorem.

2 Categories, Functors, and Natural Transformations

Definition 2.1 (Category). *A category $\mathcal{C}$ consists of objects $A, B, \dots$, sets of morphisms $\mathrm{Hom}_{\mathcal{C}}(A, B)$, identity morphisms $\mathrm{id}_A : A \rightarrow A$, and a composition operation*

$$\mathrm{Hom}_{\mathcal{C}}(B, C) \times \mathrm{Hom}_{\mathcal{C}}(A, B) \longrightarrow \mathrm{Hom}_{\mathcal{C}}(A, C), \quad (g, f) \longmapsto g \circ f,$$

such that composition is associative and identities act as units.

The definition is deliberately relational. It does not say what an object is made of; it says how objects map to each other. This is why category theory is well suited to comparing mathematical structures whose underlying material is different but whose transformation laws are similar.

Definition 2.2 (Functor). *A functor $F : \mathcal{C} \rightarrow \mathcal{D}$ sends each object $A \in \mathcal{C}$ to an object $F(A) \in \mathcal{D}$, and each morphism $f : A \rightarrow B$ to a morphism $F(f) : F(A) \rightarrow F(B)$, preserving identities and composition:*

$$F(\mathrm{id}_A) = \mathrm{id}_{F(A)}, \quad F(g \circ f) = F(g) \circ F(f).$$

Example 2.3 (Forgetting structure). *The forgetful functor*

$$U : \mathbf{Grp} \longrightarrow \mathbf{Set}$$

sends a group $G = (|G|, \cdot, e, (-)^{-1})$ to its underlying set $|G|$, and sends a group homomorphism to the same function of underlying sets. It does not choose generators; it keeps all elements and forgets the group operation.

Definition 2.4 (Natural transformation). *If $F, G : \mathcal{C} \rightarrow \mathcal{D}$ are functors, a natural transformation $\eta : F \Rightarrow G$ assigns a morphism*

$$\eta_A : F(A) \rightarrow G(A)$$

to each object $A \in \mathcal{C}$, such that for every $f : A \rightarrow B$ the square commutes:

$$G(f) \circ \eta_A = \eta_B \circ F(f).$$

Natural transformations are the morphisms between functors. Thus if a presheaf is a functor, then a morphism of presheaves is a natural transformation. This will be essential later.

3 Limits as Compatible Data

Definition 3.1 (Pullback). *Given $f : A \rightarrow C$ and $g : B \rightarrow C$, a pullback is an object $A \times_C B$ with maps $p_1 : A \times_C B \rightarrow A$ and $p_2 : A \times_C B \rightarrow B$ such that*

$$f \circ p_1 = g \circ p_2,$$

and satisfying the following universal property: if X has maps $u : X \rightarrow A$ and $v : X \rightarrow B$ with $f \circ u = g \circ v$, then there is a unique map $X \rightarrow A \times_C B$ through which u and v factor.

Example 3.2 (Set-theoretic pullback). *In **Set**,*

$$A \times_C B = \{(a, b) \in A \times B \mid f(a) = g(b)\}.$$

Thus a pullback is a space of pairs satisfying a compatibility condition.

Remark 3.3 (Why this matters for AI). *If $I \rightarrow S$ and $T \rightarrow S$ map image and text representations into a shared semantic space S , then $I \times_S T$ is the object of image-text pairs whose semantic images agree. This is not yet topos theory, but it is the kind of universal construction that one expects to exist inside a well-behaved semantic category.*

Finite limits, including terminal objects, products, equalizers, and pullbacks, are the categorical infrastructure for conjunction, equality, substitution, and context extension. This is why finite limits appear in the definition of an elementary topos.

4 Adjunctions

Definition 4.1 (Adjunction). *Let $L : \mathcal{C} \rightarrow \mathcal{D}$ and $R : \mathcal{D} \rightarrow \mathcal{C}$ be functors. An adjunction $L \dashv R$ is a natural bijection*

$$\mathrm{Hom}_{\mathcal{D}}(LC, D) \cong \mathrm{Hom}_{\mathcal{C}}(C, RD)$$

for all $C \in \mathcal{C}$ and $D \in \mathcal{D}$.

Example 4.2 (Free group and forgetful functor). *Let $F : \mathbf{Set} \rightarrow \mathbf{Grp}$ be the free group functor and $U : \mathbf{Grp} \rightarrow \mathbf{Set}$ the forgetful functor. Then*

$$F \dashv U,$$

because a group homomorphism $F(X) \rightarrow G$ is uniquely determined by a function from X to the underlying set $U(G)$:

$$\mathrm{Hom}_{\mathbf{Grp}}(F(X), G) \cong \mathrm{Hom}_{\mathbf{Set}}(X, U(G)).$$

This does not mean F and U are inverse functors. It means that maps out of a free object are equivalent to unstructured maps out of its generators.

Adjunctions occur repeatedly in topos theory. Sheafification is left adjoint to the inclusion of sheaves into presheaves. Geometric morphisms are adjoint pairs of functors. Quantifiers in internal logic can be expressed as adjoints to pullback functors.

5 Presheaves and the Yoneda Lemma

Definition 5.1 (Presheaf). *For a category $\mathcal{C}$, a presheaf on $\mathcal{C}$ is a functor*

$$F : \mathcal{C}^{op} \longrightarrow \mathbf{Set}.$$

The category of presheaves is

$$\mathbf{PSh}(\mathcal{C}) = [\mathcal{C}^{op}, \mathbf{Set}].$$

The contravariance has a simple meaning. If $U \rightarrow V$ expresses that U is a smaller context inside V , then a section over V can be restricted to a section over U :

$$F(V) \longrightarrow F(U).$$

Definition 5.2 (Representable presheaf). *For $A \in \mathcal{C}$, the representable presheaf yA is*

$$yA = \mathrm{Hom}_{\mathcal{C}}(-, A).$$

Thus $yA(X) = \mathrm{Hom}_{\mathcal{C}}(X, A)$.

Lemma 5.3 (Yoneda). *For every presheaf $F : \mathcal{C}^{op} \rightarrow \mathbf{Set}$ and every object $A \in \mathcal{C}$, there is a natural bijection*

$$\mathrm{Nat}(yA, F) \cong F(A).$$

Proof. Let $\eta : yA \Rightarrow F$ be a natural transformation. Define

$$a = \eta_A(\mathrm{id}_A) \in F(A).$$

For any $f : X \rightarrow A$, naturality for f gives

$$\eta_X(yA(f)(\mathrm{id}_A)) = F(f)(\eta_A(\mathrm{id}_A)).$$

Since $yA(f)$ is precomposition with f ,

$$yA(f)(\mathrm{id}_A) = \mathrm{id}_A \circ f = f.$$

Therefore

$$\eta_X(f) = F(f)(a).$$

So the whole natural transformation is determined by the single element $a \in F(A)$.

Conversely, given $a \in F(A)$, define

$$\eta_X(f) = F(f)(a)$$

for every $f : X \rightarrow A$. Functoriality of F implies that these maps are natural in X . The two constructions are inverse to one another. $\square$

Remark 5.4. *Yoneda says that an element of $F(A)$ is the same thing as a rule assigning compatible F -data to all generalized elements $X \rightarrow A$. This is the first precise form of the slogan: an object is understood by how all other objects map into it.*

6 Sheaves, Sites, and Sheafification

Definition 6.1 (Grothendieck topology and site). *A Grothendieck topology J on $\mathcal{C}$ specifies which families*

$$\{U_i \rightarrow U\}_{i \in I}$$

are to be regarded as covers of U . The pair $(\mathcal{C}, J)$ is called a site.

Definition 6.2 (Sheaf). *A presheaf $F : \mathcal{C}^{op} \rightarrow \mathbf{Set}$ is a sheaf on $(\mathcal{C}, J)$ if compatible local sections over every covering family glue uniquely to a global section.*

In the topological case this means: if $U = \bigcup_i U_i$, if $s_i \in F(U_i)$, and if

$$s_i|_{U_i \cap U_j} = s_j|_{U_i \cap U_j}$$

for all i, j , then there is a unique $s \in F(U)$ such that

$$s|_{U_i} = s_i.$$

Example 6.3 (Bounded continuous functions). *On $X = \mathbb{R}$, let*

$$F(U) = \{f : U \rightarrow \mathbb{R} \mid f \text{ is continuous and bounded}\}.$$

This is a presheaf but not a sheaf. The functions $f_n(x) = x$ on $(-n, n)$ are bounded and continuous and are mutually compatible, but they glue to $f(x) = x$ on $\mathbb{R}$, which is not bounded.

Definition 6.4 (Sheafification). *Let $i : \mathbf{Sh}(\mathcal{C}, J) \hookrightarrow \mathbf{PSh}(\mathcal{C})$ be the inclusion. Sheafification is a functor*

$$a : \mathbf{PSh}(\mathcal{C}) \rightarrow \mathbf{Sh}(\mathcal{C}, J)$$

left adjoint to i :

$$a \dashv i.$$

Concretely, sheafification repairs two defects. First, it identifies sections that are locally equal. Second, it adds global sections obtained by gluing compatible local data. In the bounded-continuous example, sheafification gives the ordinary sheaf of continuous functions, because every continuous real-valued function is locally bounded.

7 Grothendieck Topoi and Internal Logic

Definition 7.1 (Grothendieck topos). *A Grothendieck topos is a category equivalent to sheaves on a site:*

$$\mathcal{E} \simeq \mathbf{Sh}(\mathcal{C}, J).$$

A Grothendieck topos can be read in three ways. It is a generalized space; it is a universe of context-dependent sets; and it carries an internal logic.

Definition 7.2 (Subobject classifier). *In an elementary topos, a subobject classifier is an object Ω such that subobjects $S \hookrightarrow X$ are classified by maps*

$$\chi_S : X \rightarrow \Omega.$$

In $\mathbf{Set}$, $\Omega = \{0, 1\}$.

The internal logic of a topos interprets predicates as subobjects and truth values as maps into Ω . This matters in AI only if one has specified which objects are semantic objects, which subobjects are predicates, and which morphisms express evidence, restriction, or transformation.

7.1 Why finite limits, exponentials, and truth values appear together

The definition of an elementary topos may look artificial when first seen: finite limits, exponentials, and a subobject classifier. The point is that these three structures reproduce, categorically, the basic operations of ordinary set-based reasoning.

Finite limits supply the grammar of contexts. A product $X \times Y$ is the context in which one has an X -variable and a Y -variable. A pullback represents substitution: if a predicate or construction is defined over Y and one has a map $X \rightarrow Y$, then the pullback transports that construction to the X -context. Equalizers represent equations.

Exponentials supply function types. An exponential Y^X represents the object of maps from X to Y , characterized by the natural bijection

$$\mathrm{Hom}_{\mathcal{E}}(Z \times X, Y) \cong \mathrm{Hom}_{\mathcal{E}}(Z, Y^X).$$

This says that a Z -parametrized family of maps $X \rightarrow Y$ is the same thing as a map $Z \rightarrow Y^X$. In machine-learning language, this is the difference between treating a model family externally as many functions and treating it internally as one object of functions.

The subobject classifier supplies truth values. If $S \hookrightarrow X$ is a predicate on X , then the characteristic map

$$\chi_S : X \rightarrow \Omega$$

is the internal truth-value function of that predicate. In **Set**, the truth values are simply 0 and 1. In a sheaf topos, the truth value of a statement may be local: it may be true on some parts of a context and not on others. This is the source of the phrase “internal logic”. It does not mean that the topos is an AI reasoner. It means that the category itself supports a logical semantics.

7.2 Quantifiers as adjoints

Let $p : X \rightarrow Y$ be a morphism in a topos. Pullback along p sends a predicate on Y to a predicate on X :

$$p^* : \mathrm{Sub}(Y) \longrightarrow \mathrm{Sub}(X).$$

In the usual logical situations this functor has adjoints

$$\exists_p \dashv p^* \dashv \forall_p.$$

The left adjoint $\exists_p$ behaves like existential quantification along the fibres of p , and the right adjoint $\forall_p$ behaves like universal quantification. This explains why adjunctions are not optional decoration: they are the categorical form of logical movement between contexts.

For AI applications this is a useful warning. If one claims that a model has an internal logic, then one should say what its predicates are, what the relevant context maps are, and whether the corresponding pullback functors really have the adjoints that would justify existential or universal language.

8 Stacks

A sheaf glues set-valued local data. A stack glues category-valued local data: not only objects, but also isomorphisms between objects. This becomes relevant when local neural representations are not literally equal but equivalent up to symmetry, reparametrization, basis change, or gauge transformation.

For example, two attention heads or two hidden feature spaces may implement the same functional role after a change of coordinates. A sheaf is too strict if it requires equality. A stack-like language is designed to remember equivalences and their compatibilities.

9 A Small Worked Model: Context-Indexed Prediction

Before reading ambitious claims about LLMs, it is useful to build a small model where the words can be checked. Let $\mathcal{W}$ be the poset category of finite token windows in one fixed document, ordered by inclusion. Thus there is a unique morphism $U \rightarrow V$ precisely when $U \subseteq V$.

Define a presheaf $P : \mathcal{W}^{op} \rightarrow \mathbf{Set}$ by letting $P(U)$ be a set of allowable next-token distributions computed from the window U . If $U \subseteq V$, then the morphism $U \rightarrow V$ in $\mathcal{W}$ induces a restriction map

$$P(V) \longrightarrow P(U).$$

This map says: a prediction made using the larger window V can be examined only on the information visible in the smaller window U .

This example explains the contravariant direction. More context restricts to less context. It also explains why a presheaf is weaker than a sheaf. Suppose a large window V is covered by overlapping subwindows U_i . A family of local predictions $p_i \in P(U_i)$ may agree on overlaps but fail to come from one global prediction in $P(V)$. If this happens, P is not a sheaf for that coverage. Sheafification would formally add the missing glued predictions and identify locally indistinguishable ones.

This is a mathematical toy model, not a faithful theory of chain-of-thought reasoning. Chain-of-thought often contains provisional, inconsistent, or self-correcting statements. A sheaf condition is a consistency principle; it is not designed to model productive contradiction by itself. A better categorical model of chain-of-thought would likely need a richer base category whose morphisms record revision, contradiction, refinement, or proof search, rather than only restriction to smaller windows. This is precisely why specifying $\mathcal{C}$, J , and the intended morphisms is not a technicality: it determines what phenomena the formalism can and cannot express.

10 A Template for Turning an AI Claim into a Theorem

Many papers in this area use the same informal sentence:

“Some neural architecture lives in a topos.”

To make this sentence into a theorem one needs a sequence of mathematical choices. The following template is intentionally strict.

1. Define a category $\mathcal{C}_{\mathcal{N}}$ of contexts associated with the neural systems under study. Objects might be layers, feature spaces, architectural states, token windows, prompts, tasks, or semantic situations. Morphisms must be genuine structure-preserving maps, not analogies.
2. Define a Grothendieck topology $J_{\mathcal{N}}$. This says which families of contexts count as covers. The choice of coverage encodes the intended local-to-global principle.
3. Define the presheaves or sheaves representing the neural data. For example, a presheaf may assign to a context the set of local states, local parameters, local predictions, or local semantic assertions available there.
4. Prove the sheaf condition, or explain why sheafification is required.

5. State the preserved property. A useful theorem does not merely say that a representation exists; it says that the representation preserves composition, symmetry, invariance, semantic agreement, expressivity, or another named property.

Principle 10.1 (No free topos). *One does not obtain a topos merely by naming a complicated system. One obtains a topos by constructing a site and taking sheaves on it, or by proving the axioms of an elementary topos for a specified category.*

11 How to Read Topos-Based AI Claims

Principle 11.1 (Four-level classification). *Every claim in the topos-AI literature should be classified into one of four levels.*

1. *Classical topos theory: established definitions and theorems.*
2. *Structural representation: a neural object is encoded in a category, topos, or stack.*
3. *Semantic interpretation: categorical structure is interpreted as meaning, context, truth, or explanation.*
4. *Algorithmic consequence: an actual training, inference, robustness, or generalization result.*

Most existing topos-AI works are strong in the first two levels, suggestive in the third, and early in the fourth.

12 Lafforgue’s Program: Topoi as Semantic Completions

Lafforgue’s proposal is best read as a semantic-completion program. Start with a category $\mathcal{C}$ of meaningful objects, such as geometric, symbolic, or knowledge-bearing structures. Yoneda embeds it into a presheaf category:

$$y : \mathcal{C} \hookrightarrow \mathbf{PSh}(\mathcal{C}).$$

Choosing a Grothendieck topology J and sheafifying gives

$$\mathbf{Sh}(\mathcal{C}, J).$$

The mathematical content is the standard topos-theoretic fact that presheaf and sheaf categories supply a richer universe containing the original objects. The AI interpretation is that numerical learning should be connected back to a semantic or geometric source category. This is a programmatic claim. It becomes a theorem only after $\mathcal{C}$, J , and the intended semantic maps are defined.

The useful lesson for a learner is not that “AI is a topos”. The lesson is that a semantic theory should distinguish three layers:

$$\text{raw data} \longrightarrow \text{structured objects} \longrightarrow \text{sheaves of locally coherent interpretations.}$$

The Yoneda embedding explains how a small source category can be represented in a presheaf universe. The topology J then expresses which local pieces are allowed to count as evidence for a global interpretation. Thus the mathematical task is to choose J so that the sheaf condition matches the intended semantics, rather than merely imposing formal gluing.

Principle 12.1 (What Lafforgue’s program proves). *The established theorem is the existence and usefulness of presheaf and sheaf topoi built from a chosen site. The AI-specific part is a proposal for choosing the site so that it captures semantic structure relevant to learning.*

13 Belfiore–Bennequin: DNNs, Topoi, and Stacks

Belfiore and Bennequin propose that deep neural networks can be represented as objects in canonical Grothendieck topoi and that invariance structures in such networks can be described by stacks.

The mathematical reading is as follows. A network should not be treated merely as a map $N_\theta : X \rightarrow Y$. It has layers, states, parameters, symmetries, feature spaces, and transformations. One therefore tries to build a category of network contexts, then forms sheaf or stack structures over it. The topos is intended to be the global semantic universe of the network; stacks are intended to encode local equivalences and invariances.

This is not a proof that neural networks generalize. It is a structural representation proposal. To make it fully rigorous in a concrete case, one must state exactly:

1. what the base category of network contexts is;
2. what the covering families are;
3. what objects or sheaves represent layers, states, or parameters;
4. what equivalences the stack is required to glue.

The reason stacks enter is subtle but important. Suppose local feature representations are defined only up to an invertible change of coordinates. If one covers a network by local regions and obtains local representations E_i , then on overlaps E_i and E_j need not be equal; they may be isomorphic. A sheaf of sets would ask for equality of restrictions. A stack asks for isomorphisms on overlaps satisfying a cocycle condition on triple overlaps. This is exactly the kind of language one needs when “same representation” means “same up to symmetry”.

Therefore the strongest mathematical reading of Belfiore–Bennequin is:

deep networks should be studied not only as functions, but as structured objects
with local symmetries whose compatible descent data can be organized by topoi
and stacks.

This is a serious structural proposal. It is not, by itself, an optimization algorithm, nor a theorem that a trained DNN has learned a correct semantic stack. Those stronger claims would need additional hypotheses and proofs.

14 The Transformer Topos Proposal

The paper *The Topos of Transformer Networks* is marked withdrawn on arXiv and should not be cited as a stable theorem. Its useful idea is nevertheless clear: transformers may require a richer categorical environment than ordinary piecewise-linear network categories because attention is context-dependent and resembles higher-order computation.

The responsible mathematical question is:

Does attention-based contextual computation force one to pass from a weaker
category of neural maps to a richer completion with exponentials, subobjects,
and internal logic?

This is a research question. A proof would need a precise category of pre-transformer networks, a precise completion construction, and a theorem showing that transformer operations live in the completion but not in the smaller category.

The phrase “topos completion” should be read with care. A completion theorem usually has the following form. Start from a category $\mathcal{A}$ that lacks some operations. Construct a category

$C(\mathcal{A})$ equipped with a functor

$$\iota : \mathcal{A} \rightarrow C(\mathcal{A})$$

such that $C(\mathcal{A})$ has the desired operations and is universal with respect to functors from $\mathcal{A}$ into categories having those operations. Thus the proof is not a metaphor about richness; it is a universal property.

For transformer networks, the natural mathematical hope is that attention requires operations analogous to dependent contextual evaluation or internal function spaces. To make this rigorous, one must specify which transformer operation cannot be represented in the smaller category and then show that the completion adds exactly the required structure. Since the cited arXiv paper is withdrawn, these notes treat it as a prompt for future formalization rather than as a reliable result.

15 Mahadevan’s Proposal for Generative AI and LLMs

Mahadevan’s proposal is to use topos-theoretic universal constructions for generative AI and LLMs. The key assertion is that suitable categories of LLM-like systems may have limits, colimits, exponentials, and subobject-classifier-like structure.

This is mathematically demanding. To prove that a category $\mathcal{L}$ of LLMs is a topos, one must prove, at minimum:

1. $\mathcal{L}$ has finite limits;
2. $\mathcal{L}$ has exponentials;
3. $\mathcal{L}$ has a subobject classifier;
4. the morphisms in $\mathcal{L}$ are defined so that the above structures are meaningful.

Without such data, the statement is best read as a design principle rather than a theorem.

The design principle is still useful. Pullbacks can express semantic alignment, pushouts can express modular fusion, equalizers can express agreement of model outputs, and coequalizers can express quotienting by behavioral equivalence. These are categorical templates for model composition.

Here is a conservative way to use the idea. Let objects of a category $\mathcal{L}$ be model interfaces rather than full physical training runs, and let morphisms be interface-preserving transformations. Then finite limits and colimits can sometimes be constructed at the level of interfaces even when the space of trained weights has no clean categorical structure. This distinction matters. A category of APIs, prompts, behaviours, or semantic specifications may be mathematically manageable; a category of all trained LLMs with arbitrary fine-tuning operations is much harder to make precise.

Thus Mahadevan’s proposal is best read in two layers. The categorical layer asks which universal constructions are useful for composing generative systems. The topos layer asks whether those constructions extend all the way to a topos-like semantic universe. The first layer is easier to formalize. The second requires the finite-limit, exponential, and subobject-classifier checks listed above.

16 The Jia–Peng–Yang–Chen Survey

The 2025 survey by Jia, Peng, Yang, and Chen is not itself a theorem. It is a map of the literature. Its useful contribution is to separate established applied category theory in machine learn-

ing from the more exploratory topos-based line. Gradient-based categorical learning, Markov categories, lenses, and equivariant learning are closer to established frameworks. Grothendieck topoi, stacks, and internal logic form a more speculative layer.

The survey is especially useful as a warning about vocabulary. Not every categorical method in machine learning is topos theory. A monoidal category of processes, a Markov category of probabilistic maps, a lens category of bidirectional updates, and a sheaf topos of contextual data solve different problems. They may interact, but they should not be collapsed into one slogan.

17 What Is Proven and What Is Not

The rigorous mathematical content of these works lies mostly in the categorical language itself: presheaves, sheaves, sheafification, Grothendieck topoi, stacks, and internal logic. These are established concepts.

The AI-specific claims are mostly structural. They claim that a neural architecture can be encoded in a certain categorical environment, or that a topos-like completion provides the right semantic universe. Such claims are valuable when the encoding is faithful to the architecture and when the categorical structure explains something not visible in the ordinary function notation.

The strongest future work would turn the structural proposals into theorems of the following form:

Given a precisely defined class of neural systems $\mathcal{N}$, there is a canonical site $(\mathcal{C}_{\mathcal{N}}, J_{\mathcal{N}})$ such that the semantics of $\mathcal{N}$ is represented by specified objects, sheaves, or stacks in $\mathbf{Sh}(\mathcal{C}_{\mathcal{N}}, J_{\mathcal{N}})$, and this representation preserves a named architectural property.

18 References

- Jean-Claude Belfiore and Daniel Bennequin, [Topos and Stacks of Deep Neural Networks](#).
- Laurent Lafforgue, [Some Possible Roles for AI of Grothendieck Topos Theory](#).
- Mattia Jacopo Villani and Peter McBurney, [The Topos of Transformer Networks](#). The arXiv record marks this paper as withdrawn and requiring major revision.
- Sridhar Mahadevan, [Topos Theory for Generative AI and LLMs](#).
- Yiyang Jia, Guohong Peng, Zheng Yang, and Tianhao Chen, [Category-Theoretical and Topos-Theoretical Frameworks in Machine Learning: A Survey](#).
- Saunders Mac Lane and Ieke Moerdijk, *Sheaves in Geometry and Logic*.
- Steve Awodey, *Category Theory*.